

Lab 3 - Vic Grigoryev (vgrigory)

1. (T&W 4-1) A computer system has enough room to hold four programs in its main memory. These programs are idle waiting for I/O half the time. Assuming that all I/O operations are distributed randomly with even probability—that is, any given process is blocked waiting for I/O with probability $1/2$ at any given time in its execution—what proportion of the time will you expect the CPU to be idle?

The probability that all four programs would be idle is $(1/2)^4$, which would be $1/16$ of the time.

2. (T&W 4-4) Using the page table of Fig. 4-8 in T&W, give the physical address corresponding to each of the following virtual addresses:

(a) 20 $\rightarrow$ 0-4k virtual $\rightarrow$ 8k-12k physical $\rightarrow 20 - 0 + 8192 = 8212$

(b) 4100 $\rightarrow$ 4k-8k virtual $\rightarrow$ 4k-8k physical $\rightarrow 4100 - 4096 + 4096 = 4100$

(c) 8300 $\rightarrow$ 8k-12k virtual $\rightarrow$ 24k-28k physical $\rightarrow 8300 - 8192 + 24576 = 24684$

3. (T&W 4-8) A computer with a 32-bit address uses a two-level page table. Virtual addresses are split into a 9-bit top-level page table field, an 11-bit second-level page table field, and an offset. How large are the pages and how many are there in the address space?

Offset is $32 - 9 - 11 = 12$, so the page size is $2^{12} = 4096$ kbytes

9bit for the first table $\rightarrow 2^9 = 512$ pages in the first page table

11bit for the second tables $\rightarrow 2^{11} = 2048$ pages in the second tables

$2^9 * 2^{11} = 2^{20} = 1048576$ pages total

4. (T&W 4-14) A computer has four page frames. The time of loading, time of last access, and the R and M bits for each page are as shown below (the times are in clock ticks):

Page	Loaded	Last Ref.	R	M
0	126	279	0	0
1	230	260	1	0
2	120	272	1	1
3	160	280	1	1

(a) Which page will NRU replace?

Page 0 - Class 0 $\leftarrow$ replaced first

Page 1 - Class 2

Page 2,3 - Class 3

(b) Which page will FIFO replace?

Sorting by load time, the order is 2, 0, 3, 1, so page 2 will be replaced first

(c) Which page will LRU replace?

Sorting by time of last access, the order is 1, 2, 0, 3, so page 1 will be replaced first

(d) Which page will second chance replace?

We use the load time list but inspect the read bit of page 2, which is 1. This means the list becomes 0, 3, 1, 2. Page 0's read bit is 0 so page 0 is replaced.

5. (Based on T&W 4-19) If FIFO page replacement is used with four page frames and eight pages, how many page faults will occur with the reference string 0713272013 if the four frames are initially empty. What will be the final contents of memory?

The first four pages, 0713 are put into memory in that order because of FIFO. Now we want to reference page 2, so we remove page 0 and add page 2 - 7132. 7 and 2 are already in virtual memory so we don't need to replace anything. Now we reference page 0 which is no longer in virtual memory, so we replace 7 with 0 - 1320. 1 and 3 are in virtual memory so we do not need to replace anything. Our memory ends up as 1320 and we had 4 page faults at the beginning because the memory was empty, and 2 more page faults while accessing.

6. Repeat Problem 5 for LRU replacement.

The first four pages, 0713 are put into memory in that order because that's the order they were accessed. Like FIFO, we need to access page 2 so we replace the 0th page to get 7132. Now we access 2, 7, 2, which means we push them to the back of the line, getting 1372. Now we have to replace a page to put in page 0, so we replace 1 to get 3720. We need to access page 1 again so we replace page 3, so we get 7201. Now we need to access page 3 so we replace page 7 and we get 2013. Our memory ends up as 2013 and we had 4 page faults at the beginning because the memory was empty, and 4 more page faults while accessing.

7. (T&W 4-20) It has been observed that the number of instructions executed between page faults is directly proportional to the number of page frames allocated to a program. If the available memory is doubled, the mean interval between page faults is also doubled. Suppose that a normal instruction takes 1 microsecond, but if a page fault occurs, it takes 2001 microseconds (i.e. 2 msec to handle the fault). If a program takes 60 seconds to run, during which time it gets 15,000 page faults, how long would it take to run if twice as much memory were available?

The program took 60 seconds to run and had 15000 page faults. It takes 2 milliseconds to compensate for a page fault. $15000 \text{ page faults} \times 2 \text{ milliseconds} = 30 \text{ seconds}$ sorting through page faults. Twice the memory means that there are half less page faults, so now we get $7500 \text{ page faults} \times 2 \text{ milliseconds} = 15 \text{ seconds}$ of sorting through paging. Since 30 seconds is used for running normal instructions anyway, we add $30 + 15 \text{ seconds}$ which is 45 seconds to run the program.

8. (T&W 5-10) The Unix system has a call to read a directory entry. Since directories are just files, why is it necessary to have a special call? Can users not just read the raw directories themselves?

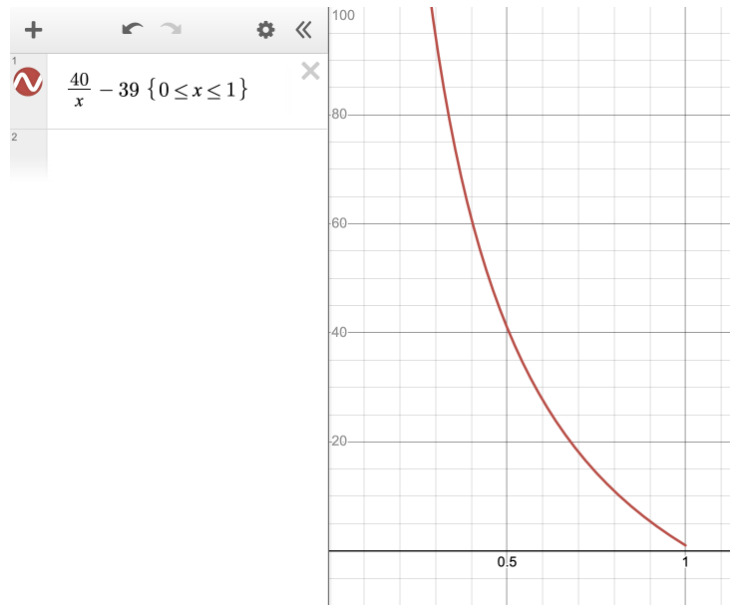
Directory entries only contain the inodes of the files they refer to along with the name of the link. This alone doesn't give much to work off of unless the user has the correct inode struct and the portion of the disk that contains the inode block to see what files the directory actually contains. Using a function to read through the directory puts this information where the user can find it without potentially reading incorrectly.

9. (T&W 5-16) It has been suggested that the first part of each unix file be kept in the same disk block as its i-node. What good would this do?

Files also begin with headers containing bits and information pertaining as to what they contain or what format they're in, which are things that are not contained in the inode. Keeping this with its inode would make it easier to fetch more detailed file properties.

10. (T&W 5-17) The performance of a file system depends on the cache hit rate (fraction of blocks found in the cache). If it takes 1 msec to satisfy a request from the cache, but 40 msec to satisfy a request if a disk read is needed, give a formula for the mean time required to satisfy a request if the hit rate is h . Plot this function for values of h from 0 to 1.0.

If the hit rate is h , then we would be hitting h times and missing $1-h$ times out of one. The total time to hit would then be $1 \cdot h = h$ msec, and missing would take $(1-h) \cdot 40 = 40 - 40h$ msec. The average would then be the total divided by h , which would be $h + 40 - 40h / h = 40 - 39h/h = 40/h - 39$ msec.



This graph works with the common sense that a larger hit rate would mean less time reading from disk.

11. (T&W 5-20) A disk has 4000 cylinders, each with 8 tracks of 512 blocks. A seek takes 1 msec per cylinder moved. If no attempt is made to put the blocks of a file close to each other, two blocks that are logically consecutive (i.e. follow one another in the file) will require an average seek, which takes 5 msec. If, however, the operating system makes an attempt to cluster related blocks, the mean interblock distance can be reduced to 2 cylinders and the seek time reduced to 100 microsec. How long does it take to read a 100 block file in both cases, if the rotational latency is 10 msec and the transfer time is 20 microsec per block?

Starting with the non adjacent blocks, we need to add the average seek, rotational latency, and transfer time, then multiply by the 100 blocks to read our file. This comes down to 5 msec + 10 msec + 0.02 msec respectively, which is 15.02 msec. Multiplying this by 100 blocks is 1502 msec to get the file.

For adjacent blocks, instead of the 5 msec average seek, we use the seek time 0.1 msec multiplied by two cylinders. This will now be 0.1 msec * 2 + 10 msec + 0.02 msec which is 10.22 msec. Multiplying this by 100 blocks will get us 1022 msec to get the file.

12. (T&W 5-24) Two computer science students, Carolyn and Elinor, are having a discussion about i-nodes. Carolyn maintains that memories have gotten so large and so cheap that when a file is opened, it is simpler and faster just to fetch a new copy of the i-node into the i-node table, rather than search the entire table to see if it is already there. Elinor disagrees. Who is right and why?

Elinor is the correct one here. Having two copies of the file in the inode table bypasses the link system which keeps all the files updated properly. When both files are opened for writing, anything that one of the files does will overwrite anything the other opened one does. Inodes and links are there to prevent this situation from happening, so putting in another inode entry for the same file means that we manually bypass the already established system for links which means both files will not properly be in sync with one another and overwrite data sections with what has been cached in memory before.